Learning from Big Data: Assignment 2

Recommending fashion items with Thompson Sampling and UCB

Dongjie (Leo) Wu Qiu
Student number: 721207

October 14, 2026

1 Introduction

ZOZOTOWN, a large Japanese fashion website, shows every visitor a row of three recommended items, and it wants to know which items to put in that row, and in which slot, to get the most clicks. With dozens of candidate items and click rates well below one percent, the site cannot afford to test every item at length, because every impression spent on a weak item is a click lost. This makes it a multi-armed bandit problem: each item is an arm and a click is the reward, so the policy has to balance trying uncertain items (exploration) against showing the items that look best so far (exploitation).

To study it we use a week of logged data from a randomised experiment that ZOZO ran in November 2019 (Saito et al., 2021). We compare Thompson Sampling (TS) and the Upper Confidence Bound algorithm (UCB), both evaluated offline with the replay method of Li et al. (2011), and extend them in four ways: TS with an informed prior and UCB with a bonus scaled to the click rates (question c), one model per user group that borrows from the pooled model (d), a minimum exposure guarantee for items (e), and discounted TS and sliding window UCB for data that change over time (g).

Both algorithms only work once their settings match how rare clicks are. With the textbook settings they stay close to random serving, whereas UCB with a small bonus raises the CTR on the 80 item file from 0.330% to 0.502%. That gain points the same way in most resamples of the week, but one week of data is too short to call it significant once the dependence between rows is taken into account. The slot does not change which items are best, and learning in large batches hurts UCB more than TS. Separate models per user group help only for the feature with a few large groups, and only when they borrow from the pooled model. The clicks that UCB earns come at a price, since it shows a handful of items and starves the rest; guaranteeing every item half its fair share of exposure ends this for about 0.13 percentage points of CTR. Finally, the daily click level shifts during the week while the item ranking stays the same, so forgetting old data does not help on this log, although it does once we make the ranking change.

2 Data

The main file has 1,356,670 impressions of 79 items over seven days (24 to 30 November 2019), with 4,720 clicks, an overall click-through rate (CTR) of 0.348%. Each row records one item shown in one slot (1 = left, 2 = centre, 3 = right), whether it was clicked, its propensity score and four hashed user features. Two smaller files contain the same log restricted to 10 items (172,400 rows, CTR 0.280%) and 20 items (345,469 rows, CTR 0.255%); the 10 item file has no user features. We use the 80 item file for all main results and the 10 item file as an easier setting with fewer arms (code Table 1). The 80 item file has 79 items because one of the 80 items that the logging policy drew from never appears in the log (the item ids run from 1 to 79).

Three facts about the data shape the rest of the analysis. First, the logging policy was uniform random. Every row has propensity $0.0125 = 1/80$ (code Table 2). The brief mentions that ZOZO also ran a Bernoulli TS policy, but those impressions are not in this file, so uniform random serving is both the policy that generated our data and our benchmark. Uniform logging is also what makes unbiased replay evaluation possible (Section 3.2).

Second, items differ but positions hardly do. Item CTRs range from 0.115% to 0.784% (Figure 1), so the best item is clicked almost seven times as often as the worst. A bandit can exploit that spread. The three positions have almost the same CTR: 0.354%, 0.349% and 0.341% (code Table 3).

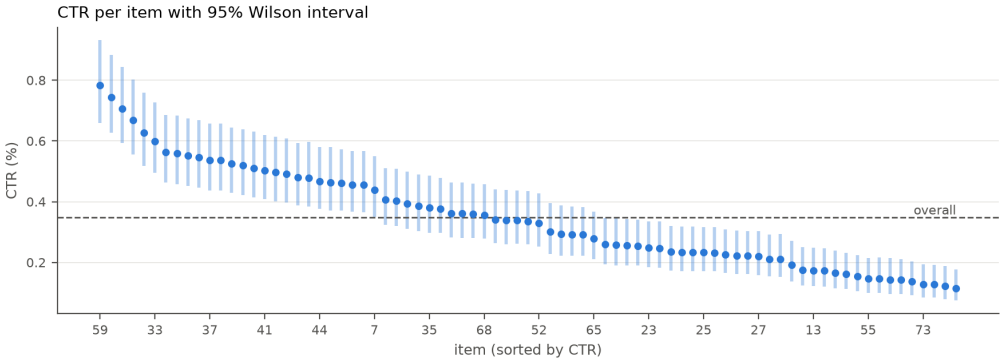


Figure 1: CTR per item in the 80 item file with 95% Wilson intervals, sorted from best to worst; the dashed line is the overall CTR (code Figure 1).

Third, one user code covers a third of the data. In all four user features the same code covers exactly 479,757 rows (35%), which suggests these are visitors without a known profile (code Table 5). Several feature levels are tiny, some with fewer than 100 rows. This matters for questions (d) and (e).

Daily CTR moves between 0.303% and 0.382% (code Table 6, Figure 2); we test in Section 9 whether this is more than noise.

3 Method

3.1 Problem setup

There are K items (arms). When item k is shown it is clicked with an unknown probability θ_k. ZOZO assumes that the click depends only on the item and its position (Saito et al., 2021); Section 4 shows that the position has no detectable effect on which items are best, so the main policies use one θ_k per item. At every impression t a policy chooses an item π_t and aims to maximise the expected number of clicks, or equivalently to minimise the regret

$$R_T = \sum_{t=1}^{T} \left(\theta^* - \theta_{\pi_t}\right), \qquad \theta^* = \max_k \theta_k, \tag{1}$$

the number of clicks lost against always showing the best item. Exploring an uncertain item costs regret now but can reveal a better item for later; the policies below differ in how they make that trade.

3.2 Offline evaluation by replay

We cannot show items to real visitors, so every policy is evaluated on the log with the replay method (Li et al., 2011). Let the log be a sequence of events (x_t, a_t, r_t), $t = 1, \ldots, T$, in time

order, where x_t is the context (slot and user features), a_t the item that was shown and $r_t \in \{0, 1\}$ the click. A policy π walks through the log. At each event it picks an item $\pi(x_t, h_{t-1})$ based on its history h_{t-1}. If the pick equals the logged item, the event is *matched*: the click is revealed, counted, and added to the history. Otherwise the event is skipped and the policy learns nothing. The replay estimate of the policy's CTR is

$$\widehat{\mathrm{CTR}}(\pi) = \frac{\sum_{t=1}^{T} r_t \, \mathbf{1}\{\pi(x_t, h_{t-1}) = a_t\}}{\sum_{t=1}^{T} \mathbf{1}\{\pi(x_t, h_{t-1}) = a_t\}}. \tag{2}$$

Li et al. (2011) show that this estimate is unbiased when the logged item was chosen uniformly at random and independently of the click, and the events are independent. The first condition holds by design. With K arms, about one event in K matches, so both the 10 and the 80 item file leave roughly 17,000 matched events and only 45 to 130 clicks per run. One standard error of a CTR estimate is 0.04 to 0.07 percentage points, and we treat differences of that size as noise.

The independence condition holds only approximately. In the log, the same item often stays in the same slot for many consecutive rows. A deterministic policy such as UCB keeps its pick until it learns something new, so it matches one row of such a streak and then waits for the next streak of its new favourite. On the 10 item file, UCB with $c = 1$ matches 4,977 events in time order but 17,227 when the rows are shuffled (code Table 18). We report matched events next to every CTR and come back to this in Section 6.

Each stochastic policy is replayed 10 times in the main comparison and 5 times in questions (b) to (g), with seed $1 + i$ for run i. The same policy can therefore score slightly differently between sections, as UCB with $c = 0.01$ does (0.502% over 10 runs, 0.569% over the first 5), but its CTR moves by about 0.11 percentage points from run to run (code Table 16), so gaps of this size are noise. Tables report the mean CTR over runs, and the code tables add the spread across runs and the binomial standard error $\sqrt{\hat{p}(1-\hat{p})/n}$ of one run with n matched events. The *Oracle* at the top of the tables always shows the item with the highest CTR over the whole week; since no real policy has that information, it is an upper bound for any policy that shows everyone the same item.

3.3 Thompson Sampling

TS keeps a Beta belief about the click probability θ_k of every item k. Starting from the prior $\mathrm{Beta}(a_0, b_0)$, after s_k clicks and f_k non-clicks the posterior is

$$\theta_k \mid \text{data} \sim \mathrm{Beta}(a_0 + s_k,\ b_0 + f_k), \tag{3}$$

by Beta-Bernoulli conjugacy. At every event TS draws one value $\tilde{\theta}_k$ from each posterior and shows $\arg\max_k \tilde{\theta}_k$; after a matched event it adds one to s_k or f_k (Russo et al., 2018). Items with an uncertain posterior sometimes draw a high value and get tried, so exploration comes from the posterior itself. The prior can be read as $a_0 - 1$ imagined clicks and $b_0 - 1$ imagined non-clicks with mean $a_0/(a_0 + b_0)$. The default $\mathrm{Beta}(1, 1)$ is flat over $[0, 1]$.

3.4 Upper Confidence Bound

UCB1 (Auer et al., 2002) first shows every item once and then shows the item with the highest index

$$U_k = \hat{\mu}_k + c\sqrt{\frac{2 \ln n}{n_k}}, \tag{4}$$

where $\hat{\mu}_k$ is the item's CTR so far, n_k the number of times it was shown and n the total number of matched events. The second term is an optimism bonus that shrinks as an item is shown more. The textbook version has $c = 1$; we treat c as a tuning parameter. Unlike TS, UCB

is deterministic given its history: the same history always gives the same choice. Its runs still differ, because in the first round every item is tried once in random order and ties between equal indices are broken at random.

3.5 Measures

Besides the CTR we report the lift over Random, $100(\widehat{\text{CTR}}/\widehat{\text{CTR}}_{\text{Random}} - 1)$, the regret per 10,000 matched events, $10^4(\widehat{\text{CTR}}_{\text{Oracle}} - \widehat{\text{CTR}})$, which estimates the regret (1) per 10,000 impressions with the Oracle CTR in place of θ^*.

To judge whether a lift is real we use an hour-block bootstrap instead of a textbook test. A two-proportion z-test against Random assumes independent rows, and Section 9 shows that rows within the same hour are not independent. One bootstrap resample draws the 168 hours of the week with replacement and puts them one after the other in the drawn order, so the streaks inside each hour stay intact. Random, TS with Beta$(1, 300)$, UCB with $c = 0.01$ and per group TS are each replayed once on the resample (seed $1 + b$ for resample b), and we record each policy's lift over the Random run on the same resample. With 100 resamples, the 2.5th and 97.5th percentiles of these lifts give a 95% interval. The two kinds of spread answer different questions. The spread across runs measures only the policy's own randomness on a fixed log. The bootstrap also lets the data vary, as another week with the same hourly dependence might, so its intervals are wider. Because Random is itself one noisy run in each resample, the intervals are on the conservative side.

4 Question (a): comparing the policies

4.1 Main comparison

Table 1 compares the policies on both files. The defaults are TS with a flat prior and UCB with $c = 1$. The tuned versions use the settings found in question (c): a prior Beta$(1, 300)$, whose mean of 0.33% is close to the overall CTR, and $c = 0.01$. We use Beta$(1, 300)$ rather than the best value in the grid, Beta$(1, 1000)$, because a prior centred on the site's average CTR can be set without looking at the results. Per group TS, with one model per level of `user_feature_0`, is explained in Section 7.

Table 1: Replay CTR of the policies, 10 runs each (code Table 13). CTR, lift, CI and > 0 are in percent. Lift is relative to Random, regret is the number of clicks lost against the Oracle per 10,000 matched events. The 95% interval (CI) for the lift and the share of resamples with a positive lift (> 0) come from 100 hour-block bootstrap resamples (code Table 13b), computed for the tuned policies only. Tuned settings come from question (c) and were chosen on the same log.

	80 items						10 items			
Policy	Matched	CTR	Lift	Regret	95% CI	> 0	CTR	Lift	95% CI	> 0
Random (logging)	17,192	0.330	0.0	45.4			0.261	0.0		
TS, Beta$(1, 1)$	17,109	0.377	14.1	40.7			0.380	45.4		
TS, Beta$(1, 300)$	17,077	0.413	25.0	37.1	$[-24, 66]$	73	0.395	51.0	$[-7, 134]$	94
UCB, $c = 1$	10,757	0.408	23.4	37.6			0.330	26.3		
UCB, $c = 0.01$	11,169	0.502	52.0	28.2	$[-17, 150]$	83	0.398	52.3	$[-15, 157]$	93
Per group TS	17,073	0.443	34.2	34.1	$[-19, 82]$	83				
Oracle (best item)	16,325	0.784	137.3	0.0			0.525	100.9		

Random replays at 0.330% on the 80 item file, close to the 0.348% of the full log, and the Oracle replays at exactly the CTR of the best item. Both checks confirm that the evaluator works (code Table 7).

The two files behave differently. With 10 items every learning policy except UCB with $c = 1$ beats Random by 45 to 52%, and TS shows the best item in 31.5% of its matched events against 10% under Random (code Table 9). With 79 items the same 17,000 matched events are spread over eight times as many arms, about 217 per item and on average less than one click each. Default TS then barely moves away from uniform serving: its gain of 0.047 points is below one standard error of the difference (0.064). The flat prior is the reason. After n non-clicks its posterior mean is $1/(n + 2)$, so after 217 shows an item without clicks still looks like 0.46%, above the true CTR of most items, and TS keeps exploring until the data runs out. Figure 2 shows the result: the TS curve separates from Random early and then stays flat, far below the Oracle.

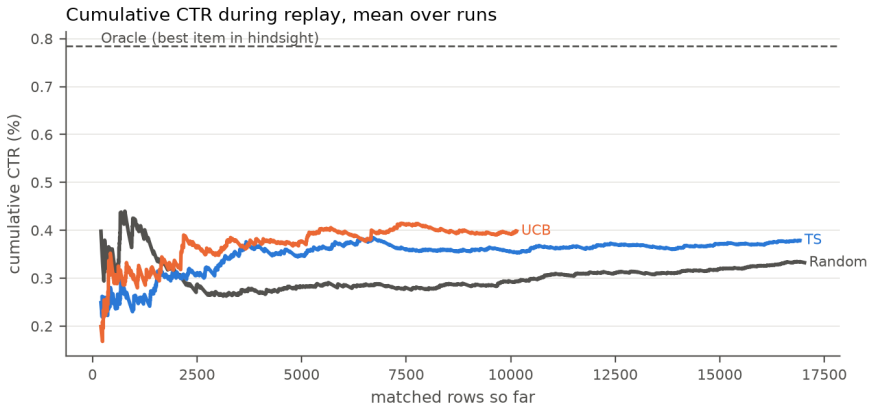


Figure 2: Cumulative CTR during replay on the 80 item file, mean over runs, for TS, UCB ($c = 1$) and Random; the dashed line is the Oracle (code Figure 3).

Tuning changes the ranking. With the informed prior TS reaches 0.413%, and UCB with $c = 0.01$ reaches 0.502%, the largest lift on the 80 item file at 52%. A textbook z-test would call this significant, but the hour-block bootstrap interval includes zero, and the same holds for every tuned policy on both files (Table 1). The lifts are positive in most resamples, so the direction is clear, but one week of data in which rows depend on their neighbours cannot pin down their size.

With $c = 1$ the bonus in (4) is about 0.2 after a few thousand events, around 70 times the size of the CTRs it is added to. UCB then mostly rotates through the items, showing each about 10% of the time on the 10 item file and about 1.3% on the 80 item file, as Random does (code Table 9). Its 0.408% on the 80 item file rests on only 10,757 matched events and is only about one standard error above Random. Even the best policy leaves about 60% of the Oracle's advantage over Random unused: one week of uniformly logged data has too few clicks to find the best of 79 items.

4.2 Where to place the items

The brief also asks where to place the items. If the best item depended on the slot, a policy would have to learn a separate ranking for the left, centre and right position, but the data give no sign of that. The five best items in each position largely overlap (code Table 10). A likelihood ratio test of an item by position interaction does reject at the textbook level, but it assumes independent rows. When we swap the three position labels between slots at random within every hour, which keeps the streaks intact but removes any real link between slot and item, the placebo statistics are just as large (permutation $p = 0.38$, code Table 11). In line with this, one TS model per position does no better than a single model for all slots (code Table 12).

The practical answer is therefore to show the best items, in any slot, and not to spend data on learning a ranking per slot.

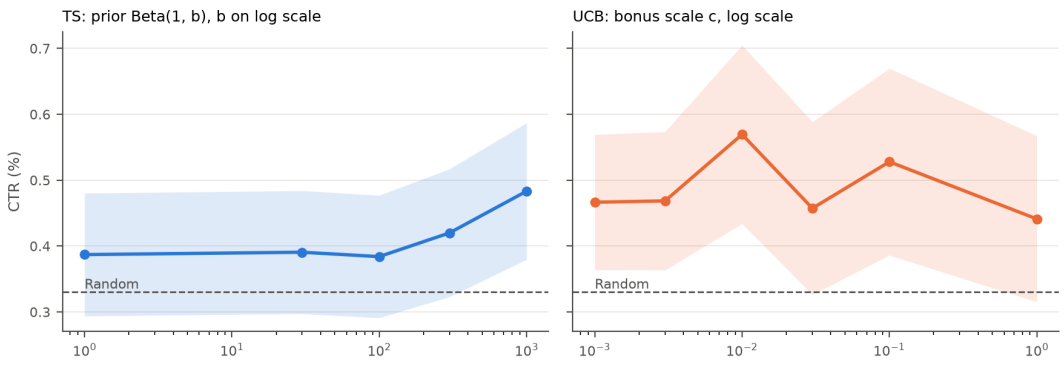


Figure 3: CTR for TS with prior Beta$(1, b)$ (left) and UCB with bonus scale c (right) on the 80 item file, 5 runs per setting, with ± 1.96 binomial standard errors; the dashed line is Random (code Figure 5, code Tables 15 and 16).

5 Question (b): batch size

A production system does not update after every impression; it collects results and updates in batches. With batch size B the policy keeps its beliefs fixed for B matched events and then processes all of them at once. We vary B from 1 to 5,000 for TS and for UCB with $c = 0.01$ (Table 2).

Table 2: CTR (%) for different batch sizes, 5 runs each (code Table 14). Random replays at 0.261% (10 items) and 0.330% (80 items).

Batch size	1	10	100	500	1,000	5,000
TS, 10 items	0.405	0.392	0.343	0.337	0.365	0.353
UCB $c = 0.01$, 10 items	0.425	0.403	0.421	0.348	0.368	0.274
TS, 80 items	0.387	0.353	0.370	0.316	0.356	0.366
UCB $c = 0.01$, 80 items	0.569	0.376	0.471	0.465	0.333	0.366

On the 10 item file both policies lose clicks as batches grow, but in different ways. TS loses some of its advantage and then levels off; at $B = 5{,}000$, which leaves only three or four updates in the whole week, it still gets 0.353%, well above Random. UCB hardly suffers up to $B = 100$ and then falls to 0.274%, Random level. The reason lies in the algorithms. During a batch UCB's index does not change, so it shows the same item for the whole batch and repeats a weak choice thousands of times. TS keeps drawing from its posterior and spreads the batch over several promising items. This agrees with Chapelle and Li (2011), who found that TS copes well with delayed feedback. On the 80 item file the picture is weaker. TS barely learns at any batch size, so it has little to lose, and UCB varies so much between runs (standard deviation up to 0.13 percentage points) that the drop is not clear beyond $B = 1$. The matched counts show a second effect. While UCB holds one choice during a batch it also catches the streaks in the log, and its matched events rise from 12,930 at $B = 1$ to about 17,000 at $B \geq 100$ (10 items).

6 Question (c): tuning

We vary the prior of TS and the bonus scale c of UCB (Figure 3).

For TS the prior matters most when there are many arms. On the 80 item file CTR rises from 0.387% with the flat prior to 0.420% with Beta$(1, 300)$ and 0.483% with Beta$(1, 1000)$, whose prior means of 0.33% and 0.10% are on the scale of the real CTRs. A flat prior claims that a CTR of 50% is as likely as one of 0.3%, and the data spends most of its clicks correcting

that claim. The most pessimistic prior does best because clicks are scarce. With a prior mean of 0.10% an item has to earn its exposure: an item without clicks loses out after a few dozen shows, and the matched events go to the items that do get clicked. On the 10 item file, with about 1,700 matched events per item, the data overrides any prior and all settings lie within noise of each other (0.31 to 0.42%, code Table 15).

UCB is more sensitive. On the 10 item file CTR follows a hill in c: 0.300% at $c = 1$, 0.444% at $c = 0.03$ and 0.341% at $c = 0.001$. A large bonus keeps UCB exploring; a very small one makes it commit to whichever item looked best early, which is sometimes an item that was lucky. On the 80 item file the best values are $c = 0.01$ (0.569%) and $c = 0.1$ (0.528%) (code Table 16). UCB1 was designed for rewards on the scale of $[0, 1]$; here it only works once the bonus is shrunk to the scale of the CTRs.

These best settings were chosen on the same log on which they are scored, so they flatter the tuned policies. As a fairer check we choose the setting on days 1 and 2 only and score it on days 3 to 7 (code Table 17). On the 80 item file the choice for UCB is $c = 0.1$, which scores 0.502% on the later days against 0.424% for the default and 0.378% for Random, so tuning UCB helps out of sample. The choice for TS, Beta$(1, 30)$, does not help (0.372% against 0.381%). On the 10 item file the first two days leave only about a dozen matched clicks, too few to separate the settings, and the defaults are chosen. In practice c and the prior have to be set before launch, ideally from earlier data on the site's typical CTR.

Finally, replaying the log in shuffled order removes the streaks, so UCB matches far more events (code Table 18). The learning policies then score lower in all six policy and file combinations, mostly by 0.05 to 0.11 percentage points, while Random does not move. Each gap is only about one standard error, but because it appears only for policies that adapt and always in the same direction, it suggests that replay on this log slightly favours policies that stay with one item, perhaps because clicks cluster within streaks. The ranking of the policies does not change, but absolute CTRs should be read with this caveat.

7 Question (d): one model or one per user group

So far every visitor gets the same recommendation, and Section 4 found no reason to change that across positions. User groups are a different matter. If they prefer different items, one model per group could do better, but each group model then learns from only a fraction of the data. We compare three designs for each of the four user features: one pooled TS, one TS per level of the feature starting from Beta$(1, 1)$, and one TS per level that borrows from the pooled model. In the last design the prior of item k, in every group, is

$$a_{0,k} = 1 + w_k\,\bar{p}_k, \qquad b_{0,k} = 1 + w_k\,(1 - \bar{p}_k), \qquad w_k = \min(m, N_k), \tag{5}$$

where $\bar{p}_k$ and N_k are the CTR and the number of matched events of item k in the pooled model so far and $m = 1{,}000$. A small group stays close to the pooled estimate; a large group soon outweighs the borrowed m shows with its own data. This is shrinkage towards the pooled mean. The pooled counts are built during the replay from past events only, so the policy uses no information that a live system would not have. A group's own clicks are counted in both terms, which double counts them slightly; the cap m keeps this small.

Table 3: CTR (%) of pooled and per group TS on the 80 item file, 5 runs each (code Table 19). Random gives 0.341% and pooled TS 0.387% in the same runs; one standard error is about 0.05.

	user_feature_0	user_feature_1	user_feature_2	user_feature_3
Levels	4	6	10	10
Per group TS, from zero	0.342	0.374	0.357	0.405
Per group TS, borrowing	0.494	0.388	0.362	0.395

Splitting from zero does not beat the pooled model beyond noise (0.342% to 0.405% against 0.387%, Table 3): each group model starts with no knowledge and has too few clicks to learn. Borrowing gives the largest gain for `user_feature_0`, the feature with the fewest and largest groups: 0.494% against 0.387% for the pooled model. The difference of 0.107 percentage points has a standard error of about 0.07, so $z \approx 1.5$, and it is the best of four features, which makes a gain of this size more likely by chance. We read it as suggestive, not as proven. For the features with ten levels, many of them small, borrowing makes no difference.

Within `user_feature_0` all three evaluable groups gain (code Table 20). Group 3, with 150,691 rows, is the clearest case: the pooled model lifts its CTR by 14% over Random, the per group model with borrowing by 100%, so this group prefers different items from the majority. Group 0 has 6,641 rows and about 80 matched events per run, too few to learn anything or even to measure how well it is served. One pooled model ignores the differences between groups, and fully separate models waste data. Borrowing sits in between and pays off only where groups are large enough.

We do not fit a contextual bandit such as LinUCB or logistic TS. The user features are hashed category codes with no order or meaning, so a linear model could only use them as one-hot group dummies, with item by group interactions to let preferences differ. That is a parametric version of the per group models with shrinkage above, and with about 60 to 85 clicks per run (code Table 19) it has no extra information to learn from.

8 Question (e): fairness

A policy that maximises clicks can be unfair in two ways. It can concentrate exposure on a few items, so that the other sellers are hardly seen and an item that was unlucky early never gets a second chance (fairness to items). It can also serve some user groups much better than others, usually small groups or groups whose taste differs from the majority (fairness to users).

We measure fairness to items with the Gini coefficient of the exposure shares e_k (the share of matched events in which item k was shown),

$$G = \frac{2\sum_{i=1}^{K} i\, e_{(i)}}{K \sum_i e_{(i)}} - \frac{K+1}{K}, \tag{6}$$

with $e_{(1)} \le \cdots \le e_{(K)}$, so $G = 0$ means equal exposure. We also count the items that receive less than half of the fair share $1/K$, and report the rank correlation between exposure and item CTR, which shows whether exposure follows merit. For fairness to users we compute, per level of a user feature, the CTR divided by Random's CTR in the same group (the lift), and report the worst group's lift and the gap between the best and the worst group. The lift removes differences in how often each group clicks anyway (under random serving the four groups click at 0.18% to 0.38%, code Table 5), so a policy is not counted as unfair for serving a group that rarely clicks. Groups with fewer than 500 matched events per run are left out because their lift cannot be estimated; on the 80 item file this excludes group 0.

As the fairness extension we add a minimum exposure guarantee. A wrapper around any policy promises every item at least a fraction ϕ of its fair share: whenever some item has been shown fewer than $\phi\, n/K$ times after n matched events, the wrapper shows the most neglected item; otherwise the base policy decides. We put the floor on UCB, the policy that concentrates exposure, with $\phi = 0.25$ and $\phi = 0.5$. The floor is the extension on the item side; per group TS with borrowing from question (d) is the extension on the user side, and we score both on the same measures.

Table 4: Clicks against fairness on the 80 item file, 5 runs each (code Table 21). Fair share per item is $1/79 = 1.27\%$.

		Fairness to items			Fairness to users	
Policy	CTR (%)	Gini	Items $< \frac{1}{2}$ share	Exposure vs CTR	Worst lift	Gap
Random	0.34	0.021	0	−0.06	1.00	0.00
TS	0.39	0.090	0	0.69	1.10	0.05
UCB $c = 0.01$	0.57	0.673	54	0.24	0.92	1.06
UCB + floor $\phi = 0.25$	0.57	0.626	50	0.28	0.81	1.21
UCB + floor $\phi = 0.5$	0.44	0.352	0	0.54	1.08	0.25
Per group TS, borrowing	0.49	0.157	0	0.67	1.28	0.72

The policies sit at different points of the trade-off (Table 4, Figure 4). UCB with $c = 0.01$ earns the most clicks, but it puts 41% of all exposure on three items and leaves 54 of the 79 items below half their fair share. It is also the least fair to users: it lifts group 2 by 97% and leaves group 3 slightly below Random (lift 0.92, code Table 22), because its few favourite items suit the majority. TS is close to fair on both counts: its random draws spread exposure ($G = 0.09$), exposure follows merit (rank correlation 0.69), and every group gains 10 to 15%. The price is a lower CTR. On `user_feature_0`, per group TS with borrowing gives users the best deal: the highest worst-group lift (1.28) with a moderate Gini of 0.16, because it serves each large group with its own favourites.

The floor does what it promises. The least shown item gets 0.32% of the exposure with $\phi = 0.25$ and 0.63% with $\phi = 0.5$, exactly a quarter and a half of the fair share of 1.27% (code Table 21). The weak floor changes almost nothing and even widens the gap between user groups. The floor at half the fair share does much more: no item is starved, the Gini halves, exposure follows merit more closely, and every group gains on Random (Table 4). Its price is about 0.13 percentage points of CTR, which still leaves the policy above TS and Random. Per group TS with borrowing does better still on CTR, Gini and worst group lift, although its gap between groups is wider. For ZOZO the floor is a simple guarantee that can be added to any policy, while personalisation is the cheaper way to be fair to both sides, but only for `user_feature_0`, as Table 5 shows.

The brief asks us to treat all four user features as sensitive variables. Table 5 repeats the user measures for every feature, with the same runs for the policies that ignore the features and a per group TS that splits on the feature being judged.

Table 5: Worst group lift over Random and gap between the best and the worst group, for every user feature, 80 item file, 5 runs each (code Table 22). Groups with fewer than 500 matched events per run are left out; the number of groups used is shown under each feature.

	`user_feature_0` 3 groups		`user_feature_1` 4 groups		`user_feature_2` 7 groups		`user_feature_3` 6 groups	
Policy	Worst	Gap	Worst	Gap	Worst	Gap	Worst	Gap
TS	1.098	0.054	1.081	0.745	1.040	0.686	1.098	0.224
UCB $c = 0.01$	0.917	1.056	1.487	2.308	1.244	2.757	1.177	1.699
UCB + floor $\phi = 0.25$	0.805	1.207	1.479	1.964	1.125	3.007	1.084	1.407
UCB + floor $\phi = 0.5$	1.076	0.253	1.224	1.288	0.898	0.869	0.735	1.849
Per group TS, borrowing	1.284	0.717	0.910	0.295	0.793	0.545	0.611	1.218

The UCB policies serve the groups most unequally on every feature: plain UCB has gaps of 1.06 to 2.76, and the floor does not reliably narrow them (the largest gap per feature is always plain UCB or one of its floor versions). Only for `user_feature_0` does plain UCB leave a group below Random; for the other features every group gains under it, just by very different amounts. The floor at half the fair share even pushes one group below Random on `user_feature_2` and

`user_feature_3` (0.90 and 0.74), so the item-side guarantee does not carry over to users. Per group TS with borrowing narrows the gap against UCB for every feature, but it raises the worst group only for `user_feature_0`. For the other three it leaves one group below Random (worst lift 0.91, 0.79 and 0.61): with more and smaller groups, the group models follow noise, which matches the lack of a CTR gain in Section 7. TS is the only policy under which every group of every feature gains on Random. Some of these lifts rest on few clicks (the smallest groups used have about one click per run under Random), so single values should not be over-read.

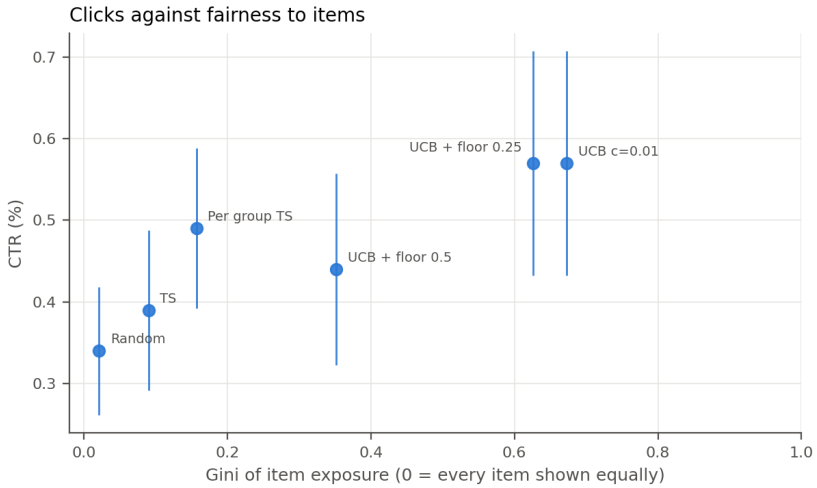


Figure 4: CTR against the Gini of item exposure on the 80 item file; error bars are ±1.96 binomial standard errors (code Figure 6).

9 Question (f): is the data stationary?

Two kinds of change matter differently for a bandit. If all items become more or less popular together, the best item stays the best and a stationary policy still picks correctly. If the ranking of the items changes, a policy that averages over the whole past keeps showing yesterday's winner. We test both on the 80 item file.

All textbook tests assume independent rows, and the streaks in the log violate that. We therefore calibrate every test with a block permutation placebo: the 168 hours of the week are dealt out at random into placebo days (or halves), keeping each hour intact, and the test statistic is recomputed 200 times. A placebo split keeps the dependence between nearby rows but contains no real trend, so it shows how large the statistic gets by chance. The permutation p-value is the share of placebo statistics at least as extreme as the real one.

Table 6: Stationarity tests on the 80 item file (code Tables 23 to 25). Placebo values come from 200 hour block permutations.

Test	Statistic	Textbook p	Placebo mean	Placebo 95%	Perm. p
Equal daily CTR, χ^2 (6 df)	27.5	0.0001	12.4	24.4	0.015
Item ranking, halves (Spearman ρ)	0.772		0.747	0.684[a]	0.74
Item × half, LR (78 df)	136.8	0.00004	166.4	207.1	0.89
Item × day, LR (468 df)	928.7	$< 10^{-6}$	972.4	1,057.8	0.82

[a] 5th percentile, since a change in the ranking lowers ρ.

The textbook tests all reject stationarity, but the placebo values show why they cannot be trusted here: even random groupings of hours give a χ^2 of 12.4 on average instead of the 6 expected under independence (Table 6). After calibration the results split. The overall daily

CTR does change: the real χ^2 of 27.5 beats 98.5% of the placebo values (permutation $p = 0.015$), with Wednesday at 0.303% and Tuesday at 0.382%. The item ranking does not. The rank correlation of item CTRs between days 1 to 3 and days 4 to 7 is 0.772, a typical value for a placebo split of hours (mean 0.747, $p = 0.74$; Figure 5). The likelihood ratio tests of an item by period interaction in a logistic model of the click give statistics below the placebo mean ($p = 0.89$ by half week and $p = 0.82$ by day). In the day-level test 23% of the cells have fewer than five expected clicks, which is one more reason to rely on the permutation version.

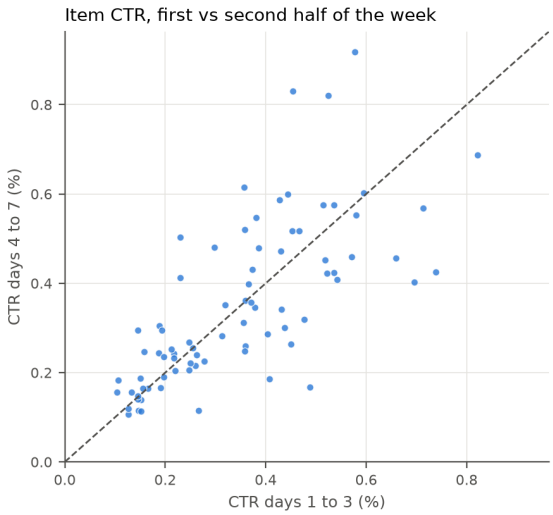


Figure 5: CTR of every item in days 1 to 3 against days 4 to 7, 80 item file; points on the diagonal behave the same in both halves (code Figure 7).

The data is therefore non-stationary in level but stationary in ranking. The level shift does not change which item is best, so it does not hurt a bandit that only needs the ranking.

10 Question (g): policies for a changing environment

If the ranking did change, the policies would have to stop weighting old and new evidence equally. We implement two such changes and outline a third.

Discounted TS (Raj and Kalyani, 2017) multiplies all counts by $\gamma < 1$ after every matched event before adding the new result,

$$s_k \leftarrow \gamma s_k + r\,\mathbf{1}\{k = a\}, \qquad f_k \leftarrow \gamma f_k + (1 - r)\,\mathbf{1}\{k = a\}, \tag{7}$$

and samples from $\text{Beta}(a_0 + s_k, b_0 + f_k)$. An event that is n updates old keeps weight γ^n, a memory of about $1/(1-\gamma)$ updates; we use $\gamma \in \{0.9999, 0.999, 0.995\}$. Sliding window UCB (Garivier and Moulines, 2011) computes $\hat{\mu}_k$ and n_k in (4) from the last W matched events only, with $W \in \{5{,}000, 2{,}000, 1{,}000\}$ and $c = 0.01$. The third option is change point detection, which monitors each item's recent CTR and restarts the policy when it departs from the long-run value. With 45 to 130 clicks per run spread over 10 to 79 items, a detector would have far too little data per item to tell a real change from noise, so we do not implement it.

Because the ranking in our data is stable (Section 9), we also build a log in which it is not: from the middle of the week the item labels are reversed by rank, so the rows of the worst item now carry the label of the best item, the second worst that of the second best, and so on. The logging stays uniform, so replay remains valid. The cost of the flip is a policy's CTR in the second half of the real log minus its CTR in the second half of the flipped log.

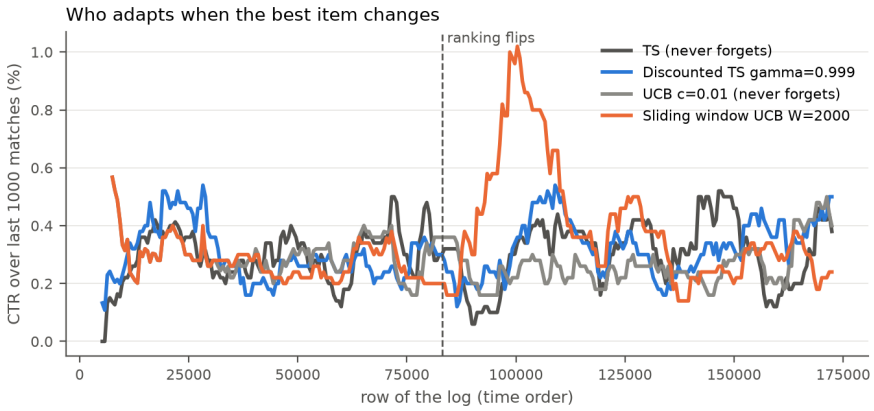


Figure 6: Moving CTR over the last 1,000 matched events on the 10 item log with the ranking flipped halfway (dashed line), mean over runs (code Figure 8). The peak of sliding window UCB rests on about ten clicks per window and overstates its level.

Table 7: Forgetting policies on the real log and on the log with the ranking flipped halfway, 5 runs each (code Tables 26 and 27). Cost of the flip in percentage points; lower means the policy adapted better.

	10 items		80 items	
Policy	CTR real log (%)	Cost of flip	CTR real log (%)	Cost of flip
TS (never forgets)	0.405	0.191	0.387	0.048
Discounted TS $\gamma = 0.9999$	0.298	−0.001	0.374	−0.066
Discounted TS $\gamma = 0.999$	0.334	0.035	0.391	−0.001
Discounted TS $\gamma = 0.995$	0.330	0.012	0.351	0.022
UCB $c = 0.01$ (never forgets)	0.425	0.153	0.569	0.103
Sliding window UCB $W = 5{,}000$	0.457	0.071	0.567	0.106
Sliding window UCB $W = 2{,}000$	0.409	0.054	0.513	0.053
Sliding window UCB $W = 1{,}000$	0.361	−0.062	0.482	0.056

On the real log, forgetting costs clicks or at best changes nothing (Table 7). On the 10 item file discounted TS loses 0.07 to 0.11 percentage points against plain TS, about 1 to 1.7 standard errors of the difference, and the shortest window costs UCB 0.06. Throwing away old clicks does not pay when the ranking is stable and clicks are scarce. On the flipped log the forgetting policies adapt better. The flip costs plain TS 0.19 percentage points on the 10 item file and every discounted version at most 0.04. On the same file sliding window UCB pays between nothing and half of the 0.15 that plain UCB pays. Figure 6 shows the mechanism: plain TS drops to about 0.1% after the flip and needs around 15,000 rows to recover, while the forgetting policies dip less and recover sooner. On the 80 item file the effects are smaller, since TS had learned little before the flip and has little to unlearn. Discounted TS with $\gamma = 0.9999$ is the only version with a noticeable gain there, and the longest window does not help UCB at all (cost 0.106 against 0.103).

The choice of model therefore depends on the type of change. On the ZOZO data, with a level shift and a stable ranking, plain TS or UCB is the better choice. Forgetting is the right tool only when evidence suggests that the ranking moves, and its memory length should then be set as long as the speed of change allows.

11 Limitations and conclusion

Three caveats qualify these results. First, replay leaves about 17,000 matched events and fewer than 130 clicks per run, so most differences between policies are within one or two standard er-

rors. Once the dependence between rows is respected, not even the 52% lift of UCB is significant: the hour-block bootstrap puts it between −17% and 150%. The data support the direction, with the tuned policies ahead of Random in most resamples, but not the size of the gain. Second, replay assumes independent events, and the streaks in the log break that assumption. They cut the number of matched events for deterministic policies, and the shuffled replay suggests that they also raise the CTR of policies that stay with one item by up to about 0.1 percentage points (Section 6). This does not change which policies come out ahead, but it makes absolute levels less certain. Third, the tuned settings were chosen on the same week of data. The out of sample check supports tuning UCB but not tuning TS. Beyond these, the data cover a single week that includes Black Friday (29 November), so the stationarity results may not carry over to other periods. The three slots are evaluated one at a time, because the log does not record which three items were shown together. And the user features are hashed, so we can say that small groups are served worse, but not who they are.

For ZOZOTOWN our advice is to show the items with the highest click rate, in any slot, since the slot does not change which items are best, and to learn them with a bandit whose exploration is scaled to how rare clicks are. The textbook settings of both algorithms stay close to random, so we recommend UCB with c between 0.01 and 0.1. With $c = 0.01$ it lifts the CTR by 52% over random serving on the 80 item file, although that setting was chosen on the same week, so the figure is optimistic and its bootstrap interval is wide. The out of sample check supports the range: $c = 0.1$, chosen on the first two days only, still beats the default and Random on the remaining five (Section 6). TS is the safer choice when settings cannot be tuned in advance or updates arrive in large batches, and it spreads exposure more evenly over items and user groups. If UCB is used, an exposure floor of half the fair share keeps every item visible at a cost of about 0.13 percentage points of CTR, and group models that borrow from the pooled model add clicks where groups are large. The item ranking was stable within the week, so there is no need to forget old data, but a production system should monitor the ranking and switch to a forgetting policy once it starts to move.

References

Auer, P., Cesa-Bianchi, N., and Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47, 235-256. https://doi.org/10.1023/A:1013689704352

Chapelle, O., and Li, L. (2011). An empirical evaluation of Thompson Sampling. *Advances in Neural Information Processing Systems 24 (NIPS 2011)*. https://papers.nips.cc/paper/2011/hash/e53a0a2978c28872a4505bdb51db06dc-Abstract.html

Garivier, A., and Moulines, E. (2011). On upper-confidence bound policies for switching bandit problems. In *Algorithmic Learning Theory (ALT 2011)*, Lecture Notes in Computer Science 6925, 174-188. https://doi.org/10.1007/978-3-642-24412-4_16

Li, L., Chu, W., Langford, J., and Wang, X. (2011). Unbiased offline evaluation of contextual-bandit-based news article recommendation algorithms. In *Proceedings of the Fourth ACM International Conference on Web Search and Data Mining (WSDM 2011)*, 297-306. https://doi.org/10.1145/1935826.1935878

Raj, V., and Kalyani, S. (2017). Taming non-stationary bandits: a Bayesian approach. *arXiv preprint* arXiv:1707.09727. https://arxiv.org/abs/1707.09727

Russo, D. J., Van Roy, B., Kazerouni, A., Osband, I., and Wen, Z. (2018). A tutorial on Thompson Sampling. *Foundations and Trends in Machine Learning*, 11(1), 1-96. https://doi.org/10.1561/2200000070

Saito, Y., Aihara, S., Matsutani, M., and Narita, Y. (2021). Open Bandit Dataset and Pipeline: towards realistic and reproducible off-policy evaluation. *NeurIPS 2021 Track on Datasets and Benchmarks*. https://arxiv.org/abs/2008.07146

A Appendix

A.1 Reproduction

All code is in Python and runs with `python main.py`, which prints every table and figure with its ID and saves them to `output/` (tables as csv and LaTeX, the full printout in `output/run_log.txt`). The random seed is fixed (`random.seed(1)`, and run i of every replay uses seed $1 + i$), so a rerun gives identical numbers. The full run takes a little over two hours. The code is submitted together with this report; it is also kept in a private GitHub repository that can be shared on request.

Table 8: Where each question's results are printed by the code.

Question	Code tables	Code figures
Data	1 to 6	1, 2
(a) comparison, placement	7 to 13, 13b	3
(b) batch size	14	4
(c) tuning	15 to 18	5
(d) user groups	19, 20	
(e) fairness	21, 22	6
(f) stationarity	23 to 25	7
(g) forgetting policies	26, 27	8

A.2 Supplementary figures

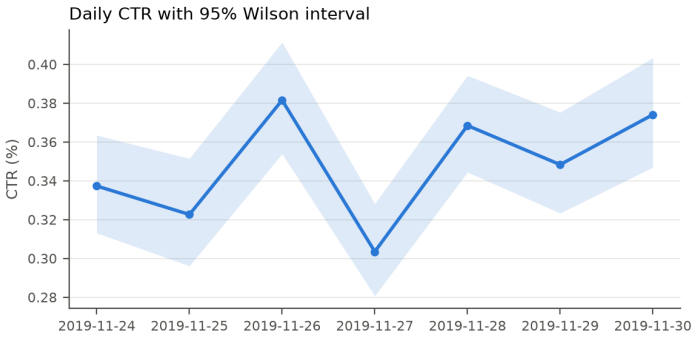


Figure 7: Daily CTR on the 80 item file with 95% Wilson intervals (code Figure 2).

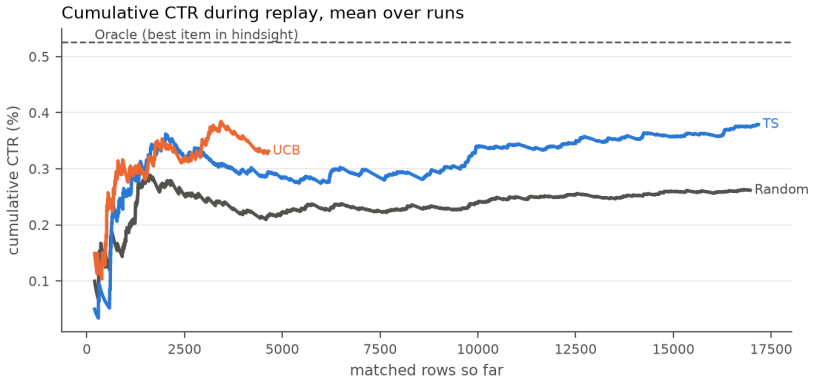


Figure 8: Cumulative CTR during replay on the 10 item file (code Figure 3).

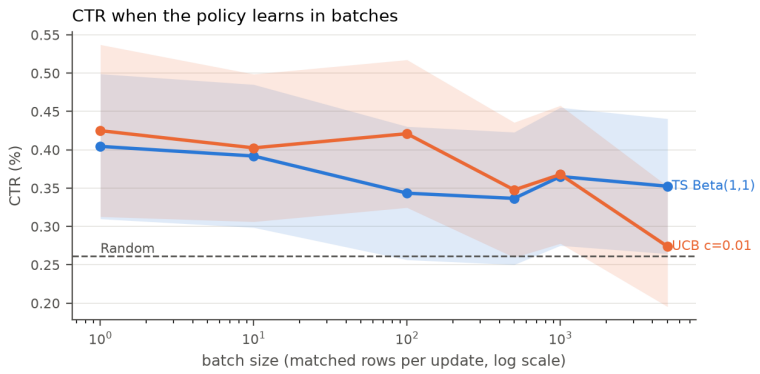


Figure 9: CTR against batch size on the 10 item file (code Figure 4).

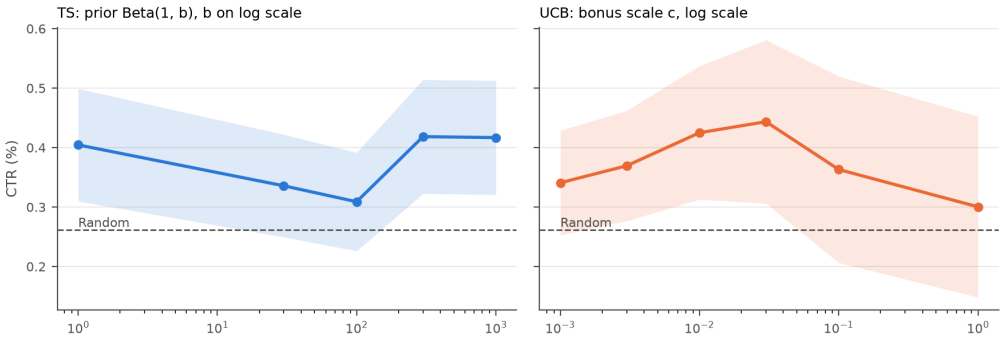


Figure 10: Tuning results on the 10 item file (code Figure 5).